

"Upload HTML" + "Prepare for Email" — Implementation Spec

This document describes a self-contained feature: letting a user **upload/paste raw HTML** for a marketing email (e.g. designed elsewhere, like Claude.ai or Figma-to-HTML) and then run a **"Prepare for email"** step that sanitizes it into something that actually renders correctly in Gmail/Outlook/Apple Mail.

Hand this whole file to Claude Code on the target project as the spec for the feature.

1. Concept

Users paste or upload arbitrary HTML for an email. This HTML is typically full of things that break in real email clients:

- Inline `<svg>` logos (not supported in Outlook)
- `<style>` blocks (stripped entirely by Gmail)
- `@import` web fonts (not supported, and slow/fragile to inline)
- Decorative `position:absolute` elements (cause dead whitespace/padding bugs in email clients)

The **"Prepare for email"** button sends this HTML to a server endpoint that:

1. Strips `@import` font declarations
2. Replaces `<svg>` logos with **merge tags** (placeholders resolved at send time)
3. Inlines all CSS from `<style>` blocks into `style="..."` attributes (via **juice**)
4. Removes leftover `position:absolute` decorative elements

The result is HTML that's safe to send, with merge tags like `{{brand_logo}}` left in place for substitution per-recipient at send time.

2. Dependencies

```
npm install juice cheerio
```

- **juice** — inlines `<style>` block CSS rules into `style="..."` attributes on each element. Required because Gmail and other clients strip `<style>` blocks / `<head>` entirely.
- **cheerio** — jQuery-like server-side HTML parsing/manipulation, used to find/replace/remove elements.

Frontend (if using a code editor for the HTML):

```
npm install @monaco-editor/react
```

3. Merge tags

The sanitizer's job is to turn raw design elements (SVG logos, etc.) into these placeholders. Adapt names to your data model (**brand** → whatever your "sender identity"/org concept is).

Tag	Expands to (at send time)
{{brand_logo}}	Full <code></code> tag for the org's logo (or text fallback if no logo)
{{brand_logo_url}}	Just the raw logo URL (for use in a custom <code></code> , e.g. footer)
{{brand_name}}	Sender/org display name
{{first_name}}	Recipient's first name (fallback: "there")
{{unsubscribe_url}}	Unsubscribe link URL

The sanitizer only ever **inserts** `{{brand_logo}}` / `{{brand_logo_url}}` (replacing SVGs). The other tags (`{{first_name}}`, `{{brand_name}}`, `{{unsubscribe_url}}`) are expected to already be present in hand-written/AI-written HTML, or added manually by the user.

4. Frontend: Upload / Paste HTML

4.1 File upload + drag-and-drop

```
function readHtmlFile(file) {
  if (!file) return
  const reader = new FileReader()
  reader.onload = (e) => set('htmlBody', e.target.result)
  reader.readAsText(file)
}

function handleHtmlDrop(e) {
  e.preventDefault()
  setHtmlDragOver(false)
  const file = e.dataTransfer.files[0]
  if (file && (file.name.endsWith('.html') || file.name.endsWith('.htm'))) {
    readHtmlFile(file)
  }
}

<div
  style={{
    border: `2px ${setHtmlDragOver ? 'dashed var(--accent)' : 'solid var(--border)'}`,
    transition: 'border-color 0.15s',
```

```

    }}
    onDragOver={e => { e.preventDefault(); setHtmlDragOver(true) }}
    onDragLeave={() => setHtmlDragOver(false)}
    onDrop={handleHtmlDrop}
  >
    <Editor
      key={editorKey}
      language="html"
      value={form.htmlBody}
      onChange={val => set('htmlBody', val || '')}
      theme="vs-light"
      options={{ minimap: { enabled: false }, fontSize: 13, lineNumbers: 'off',
wordWrap: 'on' }}
    />
  </div>

  <button type="button" onClick={() => htmlFileRef.current.click()}>↑ Upload
HTML</button>
  <input ref={htmlFileRef} type="file" accept=".html,.htm" style={{ display: 'none'
}}
    onChange={e => readHtmlFile(e.target.files[0])} />

```

4.2 Monaco editor remount trick (`editorKey`)

`@monaco-editor/react`'s `<Editor value={...}>` is a controlled component, but **changing `value` programmatically (not via user typing) does not reliably update the editor's displayed content** once it's mounted. This affects "Prepare for email" specifically: after the sanitizer returns new HTML, the editor must visibly update.

Fix: maintain an `editorKey` counter and pass it as React's `key` prop, forcing a full remount whenever the HTML is replaced programmatically:

```

const [editorKey, setEditorKey] = useState(0)

<Editor key={editorKey} value={form.htmlBody} onChange={val => set('htmlBody', val
|| '')} ... />

```

Increment `editorKey` (`setEditorKey(k => k + 1)`) every time `htmlBody` is replaced from code: after file upload, after "Prepare for email", after loading a template, etc.

4.3 Mode visibility

***Real-world gotcha:** if your app has both a drag-and-drop visual editor and a raw-HTML mode, the "Prepare for email" button only makes sense in HTML mode — it operates on raw markup. In one deployment, this button was implemented correctly from day one but went unnoticed for weeks because the user defaulted to the visual editor and never saw it. Either make HTML mode the default for the upload workflow, or add a visible hint pointing at the mode toggle when HTML is uploaded while in visual mode.*

5. The "Prepare for email" sanitizer — POST /sanitize-html

This is the core of the feature. **Order of operations matters enormously** — get this wrong and CSS-driven layout bugs (e.g. decorative absolutely-positioned blobs causing big blank gaps) won't be caught.

```
const cheerio = require('cheerio')
const juice = require('juice')

router.post('/sanitize-html', async (req, res, next) => {
  try {
    const { html } = req.body
    if (!html) return res.status(400).json({ error: 'html required' })

    let $ = cheerio.load(html, { decodeEntities: false })

    // 1. Strip @import web font declarations so juice doesn't try to fetch them.
    //    juice will attempt network requests for @import URLs (slow/fragile), and
    //    email clients don't support @import anyway.
    $('style').each((i, el) => {
      $(el).html($(el).html().replace(/@import\s+url\([^)]+\)[^;]*?\s*/g, ''))
    })

    // 2. Replace SVGs with merge tags — first SVG = header logo, second = footer
    logo,
    //    any further SVGs are removed entirely.
    //    WHY: inline <svg> elements are not supported in Outlook and many other
    clients.
    //    Pasted designs almost always use inline <svg> for the brand logo.
    let svgIndex = 0
    $('svg').each((i, el) => {
      if (svgIndex === 0) {
        $(el).replaceWith('{{brand_logo}}')
      } else if (svgIndex === 1) {
        const h = $(el).attr('height') || '28'
        $(el).replaceWith(``)
      } else {
        $(el).remove()
      }
      svgIndex++
    })
    console.log(`[sanitize] SVGs found: ${svgIndex}`)

    // 3. Run juice to inline ALL CSS from <style> blocks into style="" attributes.
    //    THIS MUST HAPPEN BEFORE STEP 4. If you remove position:absolute elements
    //    BEFORE juicing, you'll miss elements whose position:absolute rule lives
    in
    //    a <style> block (most pasted designs put layout CSS in <style>, not
    inline) —
    //    they'll survive into the final HTML and cause layout bugs (e.g. an
    invisible
```

```

    //    decorative div still occupying space and pushing content down).
    const inlined = juice($.html())

    // 4. Re-parse the now-fully-inlined HTML and remove anything with
    position:absolute.
    //    These are almost always decorative blobs/accents that work fine on a web
    page
    //    (positioned relative to a sized parent) but in email clients just leave
    behind
    //    dead space, because email clients don't honour position:absolute the way
    //    browsers do for the purposes of removing the element from document flow.
    $ = cheerio.load(inlined, { decodeEntities: false })
    let absoluteRemoved = 0
    $('*').each((i, el) => {
      const style = $(el).attr('style') || ''
      if (/position\s*:\s*absolute/i.test(style)) {
        $(el).remove()
        absoluteRemoved++
      }
    })
    console.log(`[sanitize] position:absolute elements removed:
    ${absoluteRemoved}`)
    console.log(`[sanitize] output has {{brand_logo}}:
    ${$.html().includes('{{brand_logo}}')}`)

    res.json({ html: $.html() })
  } catch (err) {
    next(err)
  }
})

```

Why each step is in this exact order

5. Strip **@import** **before** juice runs at all — juice tries to fetch **@import** URLs over the network.
6. Replace SVGs **before** juice runs — once juice inlines styles, the SVG's CSS classes get inlined styles too, making it harder to cleanly identify "the first SVG = the logo." Doing it on the raw parsed DOM first is simplest.
7. Juice **before** removing **position:absolute** — see step 3/4 comments above. This was a real bug in an earlier version: absolute elements were removed first by checking only inline **style=""**, which missed elements styled via **<style>** blocks. Those dead decorative elements then survived into the sent email and caused a visible padding/whitespace bug in Gmail. Reversing the order (juice first, then remove) fixed it.
8. Use the **case-insensitive** regex **/position\s*:\s*absolute/i** — CSS values are case-insensitive in browsers (**Position: Absolute** is valid), so the check must be too.

Debug logging worth keeping

```

console.log(`[sanitize] SVGs found: ${svgIndex}`)
console.log(`[sanitize] position:absolute elements removed: ${absoluteRemoved}`)
console.log(`[sanitize] output has {{brand_logo}}:
${$.html().includes('{{brand_logo}}')}`)

```

These three lines were the difference between guessing and *knowing* what the sanitizer did to a given input — keep them, they're cheap and invaluable when a user reports "the logo still isn't showing."

Frontend wiring

```
<button
  type="button"
  disabled={!form.htmlBody}
  onClick={async () => {
    try {
      const { data } = await api.post('/api/campaigns/sanitize-html', { html:
form.htmlBody })
      set('htmlBody', data.html)
      setEditorKey(k => k + 1) // force Monaco remount, see 4.2
      showToast('success', 'HTML prepared – SVGs replaced, CSS inlined, ready to
send')
    } catch {
      showToast('error', 'Failed to prepare HTML')
    }
  }}
>
  ✦ Prepare for email
</button>
```

6. Send-time merge tag substitution (`applyMergeTags`)

The output of "Prepare for email" still contains merge tags like `{{brand_logo}}` — this function resolves them per-recipient at send time. Without this, "Prepare for email" output would still show literal `{{brand_logo}}` text in the sent email.

```
const juice = require('juice')

function applyMergeTags(html, { firstName, brandName, unsubscribeUrl, brandLogoUrl
}) {
  const logoImg = brandLogoUrl
    ? `![${brandName}](${brandLogoUrl})
```

```
return juice(merged)
}
```

Critical gotchas

- **Never use `width="auto"` on `` tags.** It's invalid HTML — the **width** attribute must be a pixel integer or omitted. Some email clients silently collapse the image to 0x0 if it's present. Use **height="48"** + CSS **max-height/max-width** instead, and let width scale naturally.
- **Always include `border="0"`** as an HTML attribute (not just CSS) for old Outlook compatibility.
- **Run `juice()` again here**, even though the sanitizer already ran it. The merge-tag substitution inserts a fresh **/** that wasn't present during the original sanitize pass. This second pass is cheap/no-op if the HTML has no **<style>** blocks left.

7. Debug preview endpoint (no auth)

Extremely useful for diagnosing "why doesn't X show up in the sent email" issues without sending a test email and digging through an inbox. Register this **before/outside** any auth middleware so it's directly browser-accessible:

```
app.get('/api/campaigns/:id/preview', async (req, res, next) => {
  try {
    const campaign = await getCampaignWithBrand(req.params.id) // adapt to your
    data layer
    if (!campaign) return res.status(404).send('Campaign not found')

    const { name: brandName, logoUrl: brandLogoUrl } = campaign.brand
    const logoImg = brandLogoUrl
    ? `![${brandName}](${brandLogoUrl})
```

Visiting `/api/campaigns/<id>/preview` in a browser shows **exactly** what gets emailed — including whether the logo `<img>` tag is present, and whether decorative absolute-positioned elements were stripped. This single endpoint resolves most "logo/padding broken" reports faster than any other debugging step.

8. Order of implementation for Claude Code

9. Install `juice` + `cheerio` (and `@monaco-editor/react` if using a code editor).
10. Implement file upload / drag-and-drop for raw HTML (section 4.1), with the `editorKey` remount trick (4.2).
11. Implement `POST /sanitize-html` (section 5) **exactly in the order given** — the ordering is the entire point of this spec. Wire the "Prepare for email" button to it.
12. Implement `applyMergeTags` (section 6) and ensure it runs on the sanitized HTML at send/test-send time.
13. Add the public debug `GET /:id/preview` route (section 7) — use it to verify steps 3–4 produced the expected output (logo `<img>` present, no leftover absolute-positioned elements).